

TECHNICAL DESIGN, EVALUATION, AND DEVELOPMENT RECORD

RoadOps AI

Agentic Roadway Incident Decision-Support System

Proof-of-Concept Technical Documentation

Purpose
Document the business problem, architecture, agent workflow, evaluation methodology, implementation decisions, development iterations, limitations, and future work for the RoadOps AI proof of concept.

Prepared for	Kyra Solutions
Prepared by	Ved Patel
Document status	Proof of Concept - Technical Documentation
Date	September 5, 2026

RoadOps AI is an experimental decision-support system. It does not autonomously dispatch responders, control roadway infrastructure, or replace qualified human operational authority.

1. Executive Summary

RoadOps AI is a proof-of-concept agentic AI system designed to assist roadway operations personnel in assessing incidents when the initial report alone does not contain all decision-relevant context. Instead of sending a roadway incident description directly to a language model, the system uses a planner to decide what additional information may be useful, retrieves context through modular tools, grounds the assessment in a simplified operating-policy framework, and returns a structured result for human review.

The project was designed around a controlled comparison: an initial-report-only baseline and RoadOps use the same underlying language model, while RoadOps adds planning, contextual retrieval, policy grounding, structured state, and validation. This isolates the value of the system architecture rather than attributing improvement to a different model.

Key result

On the frozen 20-case A5-A24 holdout benchmark, RoadOps achieved 95.0% priority accuracy versus 5.0% for the initial-report-only baseline. Critical-factor recall improved from 23.2% to 92.2%, and guidance coverage improved from 28.3% to 81.7%. The sole RoadOps priority miss was retained and analyzed rather than tuned away.

Document map

Section	Focus
2-3	Business problem, goals, scope, and design principles
4-6	Architecture, agent workflow, tools, and policy grounding
7-9	Evaluation design, frozen holdout results, and failure analysis
10-11	Development iterations, UI/deployment, security and operational boundaries
12-14	Limitations, future work, reproducibility, and references

2. Business Problem and Project Objective

2.1 Problem statement

Roadway incident assessment can require operators to synthesize information from multiple sources. The initial report may describe an event but omit current traffic conditions, queue development, visibility, lane impact, duration, or operating guidance. A model that sees only the initial report may therefore be under-informed, inconsistent, or tempted to fill gaps with assumptions.

RoadOps explores whether an AI agent can improve this workflow by determining what contextual information is useful, retrieving that evidence, applying an explicit priority framework, and presenting a reviewable recommendation to a human operator.

2.2 Why this is an agent problem

- The information needed varies by incident; a single fixed prompt is not sufficient for every case.
- Evidence can come from different operational sources, each represented as a separate tool boundary.
- The system must distinguish between facts reported by an operator and facts retrieved from external sources.
- Policy is relevant to classification and should be provided as grounding rather than left implicit.
- The final output should remain structured, auditable, and reviewable by a human.

2.3 Working hypothesis

Hypothesis

Providing an LLM with autonomous contextual retrieval, structured reasoning, operating-procedure grounding, and validation will improve incident-assessment quality compared with an LLM operating only on the initial incident description.

3. Scope and Design Principles

Principle	Implementation choice
-----------	-----------------------

Business problem first	The project began with the operational problem and measurable comparison, not with a framework or UI.
Same-model comparison	Baseline and RoadOps use the same underlying LLM to reduce model-quality confounding.
Reproducibility	Traffic, weather, and incident-detail sources use deterministic local JSON for the POC.
Separation of concerns	Planning, retrieval, evidence representation, assessment, and evaluation are separate components.
Structured contracts	Pydantic models define tool plans, reported context, and incident assessments.
Human authority	RoadOps provides decision support only; it does not autonomously dispatch or control infrastructure.
Transparent limitations	Synthetic data, small benchmark size, planner completeness bias, and policy ambiguity are documented.

3.1 Priority classes

The simplified RoadOps POC framework supports LOW, MEDIUM, HIGH, CRITICAL, and INSUFFICIENT_INFORMATION. The priority framework is informed by public roadway incident-management guidance, but it is an independent proof-of-concept policy and is not an official FHWA classification standard.

3.2 Human-in-the-loop boundary

Recommended actions are intentionally general. RoadOps does not autonomously dispatch responders, close lanes, operate dynamic message signs, change traffic signals, or make final safety-critical decisions. A qualified human operator retains final authority.

4. System Architecture

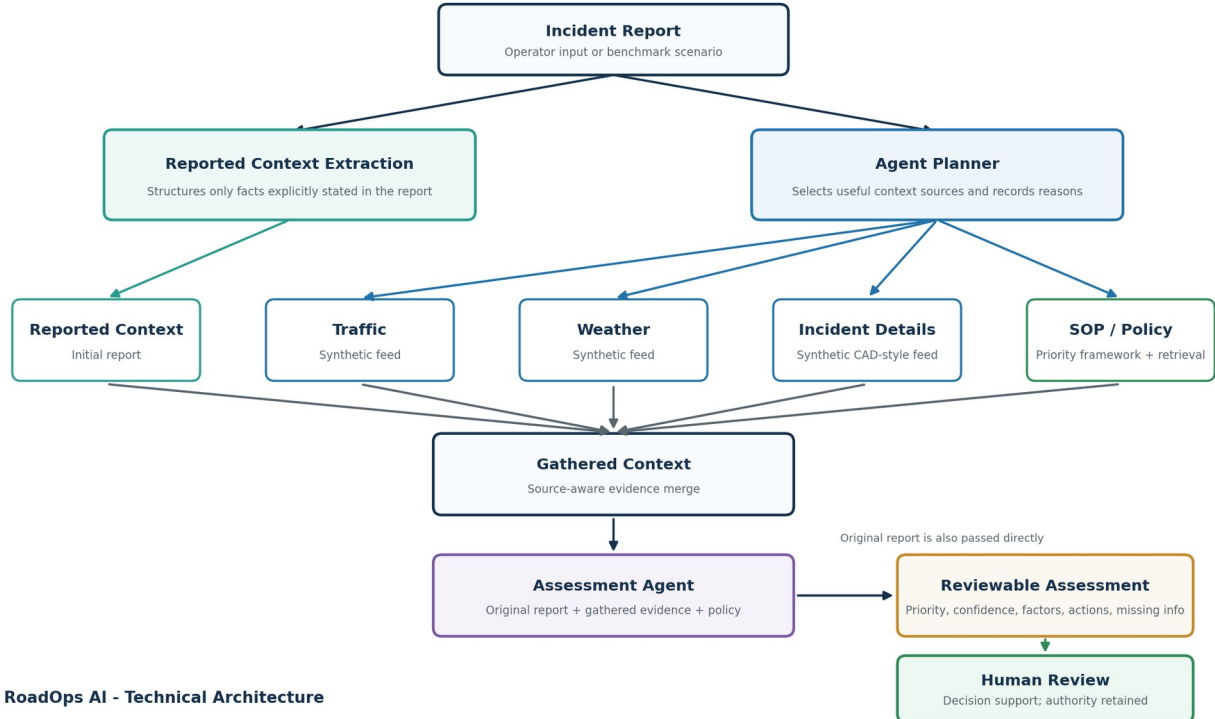


Figure 1. RoadOps architecture. The same incident report initiates reported-context extraction and agent planning. Retrieved evidence is merged into gathered context; the original report is also passed directly to the assessor.

4.1 Two parallel intake paths

The same incident report starts two complementary paths. Reported Context Extraction structures only facts explicitly stated by the operator. In parallel, the Agent Planner determines which contextual sources would meaningfully improve the assessment. This design preserves provenance: an operator-reported fact is not presented as if it came from a traffic, weather, or incident-management feed.

4.2 Evidence merge

Selected tool outputs and the structured reported context are collected in a gathered-context object. The original incident report is also passed directly to the assessment agent so relevant details are not lost through extraction or retrieval.

4.3 Assessment contract

The assessment returns a validated structure containing priority, confidence, identified factors, recommended actions, missing information, and a concise reasoning summary. This structure is used consistently by the application UI and evaluation pipeline.

5. Agent and Tool Components

Component	Role	POC implementation
Reported Context Extraction	Structures only explicitly stated incident facts.	LLM structured output into a Pydantic ReportedContext schema.
Agent Planner	Chooses potentially useful context sources and records reasons.	Structured ToolPlan with traffic, weather, incident-details, and SOP decisions.
Traffic Data Tool	Provides traffic level, queue status, and current/reference speed.	Deterministic JSON keyed by benchmark incident ID.
Weather Tool	Provides weather and visibility context.	Deterministic JSON keyed by benchmark incident ID.
Incident Details Tool	Provides operational facts such as active-lane impact and duration.	Synthetic CAD-style JSON source added after a development information-access gap.
SOP / Policy Tool	Provides complete priority framework and incident-relevant guidance.	Hybrid retrieval: always include classification framework, dynamically retrieve incident-specific sections.
Assessment Agent	Produces evidence-grounded, structured decision-support output.	Original report + gathered context -> IncidentAssessment.

5.1 Planner-assessor separation

RoadOps intentionally separates the decision about what information to retrieve from the decision about how to classify the incident. This reduces the risk that the final conclusion drives evidence gathering after the fact and makes planner behavior inspectable in Developer Mode.

5.2 Reported-context extraction rules

The extraction layer is conservative: it only structures facts explicitly stated in the report. For example, "heavy rain" may populate weather, but reduced visibility is not inferred unless visibility is explicitly stated or supplied by another evidence source. This rule prevents the intake layer from manufacturing supporting facts.

6. Policy Grounding and Priority Logic

6.1 Simplified POC guidance

- Verify available information and avoid unsupported assumptions.
- Active-lane impact generally carries more operational weight than shoulder-only impact.
- Queue development and severe speed reduction are important changing-condition indicators.
- Weather is contextual and does not automatically make a low-impact incident high priority.
- Duration is considered alongside roadway impact and changing conditions.
- CRITICAL is reserved for exceptional synthetic benchmark conditions such as full roadway closure with prolonged major disruption.

- `INSUFFICIENT_INFORMATION` is used when the available evidence is not strong enough to support a reasonable classification.

6.2 Retrieval evolution

The first SOP tool used deterministic keyword retrieval. That simple version was deliberately chosen to validate the orchestration pipeline before increasing retrieval complexity. Development testing then showed that incident-specific keyword retrieval could omit the globally relevant priority framework. The SOP tool was revised so the complete classification framework is always available while incident-specific guidance remains dynamically retrieved.

Design lesson

Global decision rules and incident-specific guidance have different retrieval requirements. The priority framework is always relevant; incident procedures can remain selective.

7. Evaluation Design

7.1 Controlled comparison

System	Information available	Purpose
Baseline	Initial incident report only; no tools, no policy retrieval.	Measure what the same model can do without the RoadOps architecture.
RoadOps	Initial report + agent planning + retrieved context + SOP grounding + structured workflow.	Measure architectural value beyond a single prompt.

7.2 Development set versus holdout set

Cases A1-A4 were used as explicit development scenarios. They were allowed to reveal architectural problems and drive fixes. After the four-tool architecture was completed, cases A5-A24 were created as a 20-case frozen holdout covering lane and congestion combinations, adverse weather, changing conditions, duration, missing information, and exceptional closures.

The holdout benchmark was frozen before final RoadOps evaluation. The baseline was executed once and preserved; RoadOps was then executed once against the frozen set and preserved. Labels and individual predictions were not changed after seeing the final results.

7.3 Metrics

Metric	What it measures
Priority accuracy	Exact match between predicted and expected priority.
Critical-factor recall	Coverage of required incident factors defined in the benchmark.
Guidance coverage	Coverage of expected decision-support guidance.
Prohibited affirmative matches	Deterministic check for benchmark-defined unsupported affirmative claims.

7.4 Independent evaluator

An early prototype asked the model to self-report unsupported assumptions. Testing showed this was unreliable: the model could describe assumptions that would be unsupported rather than identifying assumptions it had actually made. Unsupported-assumption scoring was moved into an independent deterministic evaluator.

The evaluator itself required refinement. Simple lexical matching initially confused negated or unavailable information with factual hallucinations. The final evaluator therefore separates affirmative factual assertions from missing-information, unavailable-information, and negated statements. The generated model outputs were not altered while these evaluator corrections were made.

8. Frozen Holdout Results

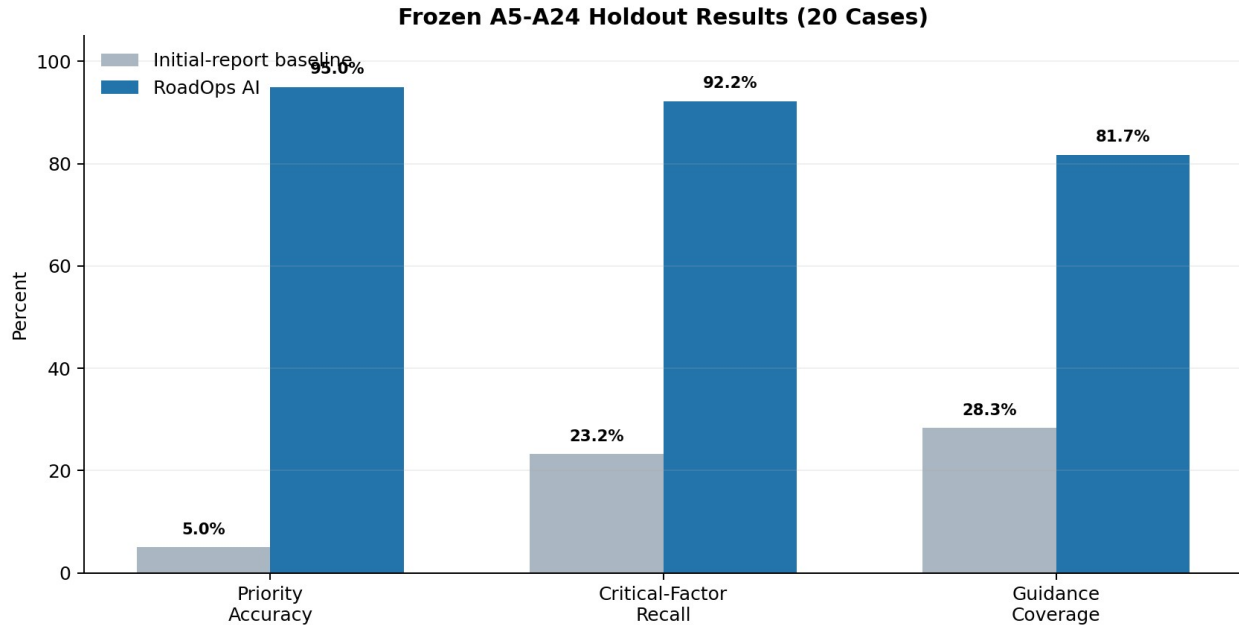


Figure 2. Frozen 20-case holdout comparison. Results are specific to the controlled synthetic benchmark and should not be interpreted as production validation.

Metric	Baseline	RoadOps	Change
Priority accuracy	5.0%	95.0%	+90.0 pp
Critical-factor recall	23.2%	92.2%	+69.0 pp
Guidance coverage	28.3%	81.7%	+53.4 pp
Prohibited affirmative matches	0	0	No increase

RoadOps correctly classified 19 of 20 holdout incidents. The baseline correctly classified 1 of 20. Because many holdout cases intentionally place decision-relevant context outside the initial report, the baseline result should be interpreted as a benchmark-specific control rather than a general estimate of LLM capability.

8.1 Planner behavior

On the frozen holdout, the planner selected all four context tools on all 20 cases: Traffic 20/20, Weather 20/20, Incident Details 20/20, and SOP 20/20. The planner therefore demonstrated dynamic structured planning but favored completeness over tool-call efficiency. Selective retrieval optimization remains future work.

8.2 Interpreting the zero prohibited-match result

The final deterministic evaluator detected zero prohibited affirmative claims under the current benchmark rules. This is not evidence that RoadOps can never hallucinate; it is a narrow statement about the frozen outputs and the benchmark-defined prohibited terms.

9. Failure Analysis: A15

The sole RoadOps priority miss occurred on A15, a prolonged shoulder incident with no active-lane blockage and no queue. The frozen benchmark expected MEDIUM because of the extended duration, while the simplified RoadOps policy strongly associates shoulder-only, non-blocking incidents with LOW.

Why the miss matters

A15 exposed a policy ambiguity rather than a simple retrieval failure: how strongly should duration increase priority when the incident remains otherwise low-impact? The benchmark label and original prediction were retained rather than revised after evaluation.

A production policy would need a clearer rule for the interaction between duration and otherwise low-impact conditions, ideally defined with transportation-domain stakeholders. Retaining the miss preserves the integrity of the frozen evaluation and provides a concrete next-step requirement.

10. Development History and Key Engineering Decisions

Development Path

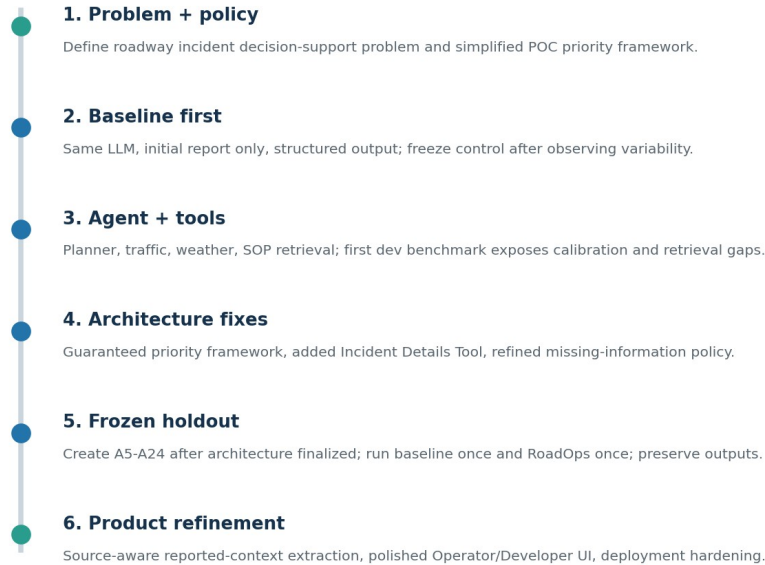


Figure 3. Development path from problem definition through frozen evaluation and post-evaluation product refinement.

10.1 Development findings that changed the architecture

Finding	Decision / change
Baseline outputs varied across repeated runs.	Freeze the original control outputs during architecture iteration to avoid control-group drift.
First four-case benchmark exposed calibration, tool-selection, and retrieval failures.	Preserve the first results before changing the system, then evaluate whether fixes improve behavior.
Keyword SOP retrieval could omit classification rules.	Always include the complete priority framework while keeping incident-specific SOP retrieval dynamic.
A4 contained lane-count/duration facts the original toolset could not observe.	Add a separate Incident Details Tool instead of leaking answer-key information into the model.
An early A4 revision became too conservative about missing fields.	Make missing information reduce confidence and remain explicit, but not automatically force INSUFFICIENT_INFORMATION when strong evidence exists.
Lexical evaluator flagged negated/unknown information as unsupported facts.	Refine scoring to focus on affirmative factual assertions and filter unavailable/negated statements.
Custom incidents were understood by the assessor but UI cards showed unavailable context.	Add source-aware Reported Context Extraction as a post-evaluation product enhancement.
Parser constructed an OpenAI client before the parent module loaded .env.	Load credentials safely inside the parser module before creating the client; a modular import-order lesson.

10.2 Post-evaluation product enhancement

Reported Context Extraction was added after the frozen A5-A24 evaluation. It improves custom-input evidence representation and UI provenance, but the published 95.0% holdout accuracy and related metrics were not rerun or attributed to this enhancement. This distinction is intentionally preserved in the documentation.

11. Application Experience, Deployment, and Security

11.1 Operator Mode

Operator Mode is intentionally simple. It emphasizes the recommended priority, confidence, contextual evidence, recommended actions, missing information, reasoning, and the human-review requirement. The goal is to surface decision-relevant information without exposing internal agent mechanics to an operational user.

11.2 Developer Mode

Developer Mode exposes the system architecture, planner decisions, selected tools, raw tool data, policy context, and evaluation results. A one-time architecture briefing appears before the developer console so a reviewer understands the evidence path and system boundaries before inspecting low-level execution details.

11.3 Deployment model

The application is designed for Streamlit deployment from a GitHub repository. Local development uses environment-based credentials; deployed credentials are stored through the hosting platform secret-management mechanism rather than committed to source control.

11.4 Security boundaries

- API credentials are excluded from version control and should never be embedded in Python or JSON files.
- The repository contains synthetic operational data, not production transportation-system credentials or feeds.
- RoadOps does not perform autonomous external actions such as dispatch, roadway control, or system writes.
- A production deployment would require authentication, authorization, rate limiting, logging, source validation, and formal operational review.

12. Limitations

Limitation	Implication
Synthetic operational data	Demonstrations are reproducible, but the POC does not validate live API behavior, coverage, or latency.
20-case holdout	Useful for controlled comparison, but too small to establish production reliability.
Simplified POC policy	Priority logic requires domain-owner validation before any operational use.
Planner completeness bias	All four tools were selected on all holdout cases, so cost/latency efficiency is not yet optimized.
LLM variability	Structured outputs and grounding reduce ambiguity but do not make model behavior perfectly deterministic.
No production system integration	The POC does not connect to CAD, traffic-management, dispatch, or roadway-control systems.
Human review required	RoadOps is decision support, not autonomous safety-critical control.

13. Future Work

- Replace synthetic traffic context with an authorized live traffic or transportation-management source while retaining a deterministic evaluation mode.
- Integrate live weather by incident location and preserve clear source provenance.
- Connect the Incident Details Tool to an authorized CAD/incident-management API in a sandboxed environment.
- Evaluate semantic policy retrieval while guaranteeing globally required classification rules remain available.
- Optimize tool selection using expected information value, latency, and cost rather than completeness alone.
- Expand the frozen benchmark with more policy edge cases, adversarial inputs, and domain-reviewed labels.
- Add per-factor provenance so each assessment factor can be traced to the initial report or a specific tool response.
- Measure end-to-end latency and API cost alongside quality metrics.

14. Reproducibility and Repository Guide

14.1 Project structure

```
roadops-ai/  
├── app.py  
├── requirements.txt  
├── agent/  
│   ├── orchestrator.py  
│   └── report_parser.py  
├── tools/  
│   ├── traffic_tool.py  
│   ├── weather_tool.py  
│   ├── incident_details_tool.py  
│   └── sop_tool.py  
├── schemas/models.py  
├── data/  
├── knowledge/roadops_sop.md  
├── evaluation/  
└── outputs/ (preserved frozen results, when included)
```

14.2 Local execution

Install dependencies from requirements.txt, set OPENAI_API_KEY in a local .env file that is excluded from version control, and start the application with:

```
python -m streamlit run app.py
```

14.3 Evaluation artifacts

For reproducibility, the repository should preserve the frozen benchmark definition and the frozen baseline/RoadOps holdout outputs used to compute the published metrics. Transient debugging outputs can remain excluded.

15. AI Assistance and Development Disclosure

Consistent with the assessment permission to use AI tools, ChatGPT was used as an implementation and documentation assistant during the RoadOps AI proof of concept. The core problem definition, system logic, agent workflow, architecture, tool pipeline, evaluation methodology, and overall project direction were developed by me. This disclosure is included to clearly distinguish those decisions from the areas where I used AI assistance.

- **Code implementation support:** ChatGPT helped generate portions of the initial Python scaffolding and assisted with implementing components based on the architecture, logic, and workflow I specified. I integrated the components into the RoadOps pipeline, tested their behavior, and iterated on the implementation as the system evolved.
- **User-interface development:** ChatGPT was used extensively to help create and refine the Streamlit user interface, including CSS/HTML styling, layout implementation, interaction ideas, and visual iterations for Operator Mode, Developer Mode, and the architecture briefing.
- **Debugging and implementation iteration:** ChatGPT was used as a troubleshooting assistant to help interpret errors and suggest possible fixes during development. This included issues involving retrieval behavior, evaluator false positives, custom-input evidence display, and environment-variable/import-order problems. I evaluated the proposed changes and incorporated the solutions that fit the system's intended logic and architecture.
- **Analysis and documentation:** ChatGPT helped summarize and organize my development findings, experiment notes, challenges, results, and engineering decisions. It also assisted with drafting, editing, and formatting portions of this technical report, README, and presentation material based on the work and results produced during development.

I remained responsible for the project's technical direction and decision-making, including the problem selection, agent and tool logic, overall pipeline, benchmark design, evaluation approach, development iterations, execution of benchmark runs, preservation of frozen outputs, failure analysis, and final conclusions. The performance numbers reported in this document come from the project's preserved evaluation artifacts; they were not generated or fabricated by ChatGPT. AI assistance was not used to retroactively change frozen holdout labels or replace unfavorable final predictions.

16. References

- [1] Federal Highway Administration (FHWA), Traffic Incident Management Program. <https://ops.fhwa.dot.gov/tim/>
- [2] FHWA, Responder and Motorist Safety. <https://ops.fhwa.dot.gov/tim/about/rms.htm>
- [3] FHWA, Traffic Management Center incident information guidance. <https://ops.fhwa.dot.gov/publications/fhwahop14022/chapter4.htm>
- [4] FHWA, Traffic incident duration / classification guidance. <https://ops.fhwa.dot.gov/publications/fhwahop10014/s5.htm>
- [5] FHWA, additional incident-duration guidance. <https://ops.fhwa.dot.gov/publications/fhwahop08057/60.htm>

These public references informed the transportation-management context of the proof of concept. The RoadOps priority framework is independently simplified for evaluation and demonstration and is not an official FHWA classification system.

Document status

This document describes the RoadOps AI proof of concept as implemented and evaluated. Frozen holdout metrics refer to the pre-reported-context-enhancement architecture; later UI and intake improvements are explicitly identified as post-evaluation product refinements.